HARNESS HACKING: MEASURING EMERGENT REWARD HACKING IN HARNESS OPTIMIZATION

Anonymous authors
Paper under double-blind review

1 PROBLEM DEFINITION AND MOTIVATION

The advent of AI agents introduces a new axis to improve the usefulness and performance of LLMs in the real world: the agent harness. Automated harness optimization allows the model itself to select prompts, tools, memory, and control logic to produce the highest evaluation score. However, this new axis of improvement also introduces new possibilities of exploiting the evaluator—i.e. reward hacking. Herein, we propose to study to what extent automatic harness optimization increases the ability or tendency of models to reward hack. Here, we define reward hacking to mean obtaining credit through any behavior that violates the task specification, such as manipulating tests or exploiting their implementation—basically, any behavior that violates the intended purpose of the test.

The motivation for this is pretty obvious. Automated harness search is increasingly becoming a practical approach to building agents. As evidenced by the July 2026 Hugging Face attack by agents in OpenAI's internal evaluations, selection for measured success may also select for unwanted behavior.

Since automated harness optimization is inherently recursive—an improved harness could be used to further improve itself—an initially incidental exploit could be retained and amplified through optimization iterations.

Another key motivation is evaluating existing benchmarks as benchmarks for Recursive Self-Improvement (RSI). RSI is often thought of as a Harness optimization problem Weng (2026) and a key problem for achieving it is building robust evaluation suites for a true "RSI Harness". We will investigate if current coding benchmarks are prone to produce exploitative behavior when optimized over to seek insight for how to build more robust benchmarks.

2 EXISTING LITERATURE

Works that has agents improve their own harnesses is widely seen, especially in the last few years. The most similar work to ours is Meta-Harness (Lee et al., 2026), which makes executable harness code the search space and gives its proposer access to prior implementations, scores, and traces. Several earlier systems optimize narrower parts of the harness. DSPy (Khattab et al., 2024) expresses an LLM pipeline as a declarative program and compiles it by optimizing prompts and few-shot demonstrations against a user-supplied metric. GEPA (Agrawal et al., 2026) adds reflective prompt evolution: it reads execution traces in natural language, proposes prompt edits, and keeps a Pareto frontier of candidates. ADAS (Hu et al., 2025) moves the search space from prompts to code, with a meta agent that programs new agents and adds them to an archive of earlier discoveries.

We plan to measure reward hacking with ImpossibleBench (Zhong et al., 2026), which builds impossible variants of tasks from two existing coding benchmarks. LiveCodeBench (Jain et al., 2025) collects competitive programming problems from LeetCode, AtCoder, and Codeforces over time to limit training-data contamination. SWE-bench (Jimenez et al., 2024) consists of real GitHub issues from popular Python repositories, each resolved by editing the codebase. ImpossibleBench mutates the unit tests of each task so that they conflict with the natural-language specification, for example by changing an expected value or by adding a test that contradicts an existing one. No correct implementation can pass the mutated tests, so the pass rate on these tasks is a direct measure of the cheating rate.

3 METHEDOLOGY

Our primary goal is to measure reward hacking as a consequence of optimizing a harness. More concretely, we will use existing harness optimization methods like Meta-Harness to optimize a harness on SWE-Bench and LiveCodeBench. Meta-Harness optimizes the following reward:

$$H^* = \arg\max_H \ \mathbb{E}_{x\sim\mathcal{X},\, \tau\sim p_M(H,x)}\, r(\tau, x),$$

For a task specific reward function r. We expect there to be instances of reward hacking as a result from optimizing over success in a benchmark, which we will measure through ImpossibleBench. Since ImpossibleBench is explicitly constructed from SWE-Bench and LiveCodeBench tasks with minor modifications, we expect any unintended "cheating" strategies to be caught by ImpossibleBench.

REFERENCES

Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. GEPA: Reflective prompt evolution can outperform reinforcement learning. In *International Conference on Learning Representations*, 2026.

Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. In *International Conference on Learning Representations*, 2025.

Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. In *International Conference on Learning Representations*, 2025.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024.

Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling declarative language model calls into self-improving pipelines. In *International Conference on Learning Representations*, 2024.

Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-harness: End-to-end optimization of model harnesses, 2026. URL https://arxiv.org/abs/2603.28052.

Lilian Weng. Harness engineering for self-improvement. *lilianweng.github.io*, July 2026. URL https://lilianweng.github.io/posts/2026-07-04-harness/.

Ziqian Zhong, Aditi Raghunathan, and Nicholas Carlini. ImpossibleBench: Measuring LLMs' propensity of exploiting test cases. In *International Conference on Learning Representations*, 2026.

4 AI USAGE DISCLOSURE

We used Claude Opus 5.5 to search for existing literature, summarizing papers, and obtaining the LaTeXcode for the Meta-Harness Objective. All writing in this report is done by the authors.